

AWS App Integration

Comprehensive Learning Guide

EventBridge · Step Functions · AppFlow · SNS · SQS · MWSA

Beginner-Friendly · In-Depth · Practical

AWS Solutions Architect Series

Version 1.0 • 2025

Step

MWSA

Airflow

Table of Contents

■ Amazon EventBridge

- Simple Introduction
- Core Concepts
- Architecture & Working
- Features
- Real-World Use Cases
- Practical Example
- Integrations
- Comparison

■■ AWS Step Functions

- Simple Introduction
- Core Concepts
- Architecture & Working
- Features
- Real-World Use Cases
- Practical Example
- Integrations
- Comparison

■ Amazon AppFlow

- Simple Introduction
- Core Concepts
- Architecture & Working
- Features
- Real-World Use Cases
- Practical Example
- Integrations
- Comparison

■ Amazon SNS

- Simple Introduction
- Core Concepts
- Architecture & Working
- Features
- Real-World Use Cases

- Practical Example
- Integrations
- Comparison

■ Amazon SQS

- Simple Introduction
- Core Concepts
- Architecture & Working
- Features
- Real-World Use Cases
- Practical Example
- Integrations
- Comparison

■ Amazon MWAA

- Simple Introduction
- Core Concepts
- Architecture & Working
- Features
- Real-World Use Cases
- Practical Example
- Integrations
- Comparison

■ Amazon EventBridge

Serverless Event Bus — Connect Everything, React to Anything

1. Simple Introduction

Amazon EventBridge is a **serverless event bus service** that makes it easy to connect applications together using data from your own apps, integrated SaaS applications, and AWS services. Think of it as the central nervous system of your cloud architecture.

Why Does EventBridge Exist?

Before EventBridge, developers had to write custom code to poll for changes, build complex pipelines, and manually wire services together. This was fragile, expensive, and hard to maintain. EventBridge solves this by providing a managed, scalable event router that decouples producers from consumers.

Real-World Analogy

■ Think of EventBridge as a Smart Office Intercom System.

When a visitor arrives (an event), the receptionist (EventBridge) announces it on the intercom. Only the relevant departments listen and respond — Security checks ID, HR logs the visit, and the host comes to greet. No one needs to know who else was notified. Everyone reacts independently.

Business Problem It Solves

Problem	Without EventBridge	With EventBridge
Service coupling	Services call each other directly — tight coupling	Services react to events — fully decoupled
Scaling	Each integration requires custom polling logic	Managed, auto-scaling event routing
SaaS integration	Custom webhooks and API polling	50+ native SaaS connectors built-in
Reliability	Failed calls cause data loss	Built-in retry and dead-letter queues
Cost	Always-on polling wastes compute	Pay only per event processed

2. Core Concepts

Understanding EventBridge requires mastering its core building blocks. Let's explore each one:

Concept	Definition	Example
Event	A JSON object describing a change in state	<code>{'source':'aws.s3','detail-type':'Object Created'}</code>

Concept	Definition	Example
Event Bus	A pipeline that receives events and routes them	Default bus, Custom bus, Partner bus
Event Source	The service or app that produces events	S3, EC2, Salesforce, your own app
Rule	A filter that matches events and routes them to targets	Route all S3 events to a Lambda function
Target	The service that receives matched events	Lambda, SQS, SNS, Step Functions, EC2
Schema	Defines the structure of an event (JSON Schema)	Ensures events have required fields
Pipe	Point-to-point integration with filtering/enrichment	DynamoDB Stream → filter → enrich → SQS
Archive	Stores all events for replay	Replay last 7 days of events after a bug
Replay	Re-process archived events	Reprocess failed order events
Partner Event Source	SaaS apps that send events directly	Zendesk, Shopify, Auth0, Stripe

Event Structure (JSON)

Every event in EventBridge is a JSON object with a standard envelope:

```
{
  "version": "0",
  "id": "abc-123-def-456",
  "source": "com.mycompany.orders",
  "account": "123456789012",
  "time": "2025-06-15T10:30:00Z",
  "region": "us-east-1",
  "detail-type": "Order Placed",
  "detail": {
    "orderId": "ORD-9999",
    "customerId": "CUST-001",
    "amount": 299.99,
    "status": "PENDING"
  }
}
```

Types of Event Buses

Bus Type	Description	Who Produces Events	Use Case
Default Bus	Every AWS account has one automatically	AWS services (S3, EC2, RDS, etc.)	React to AWS service changes
Custom Bus	You create it for your own applications	Your microservices, apps	Internal app event routing

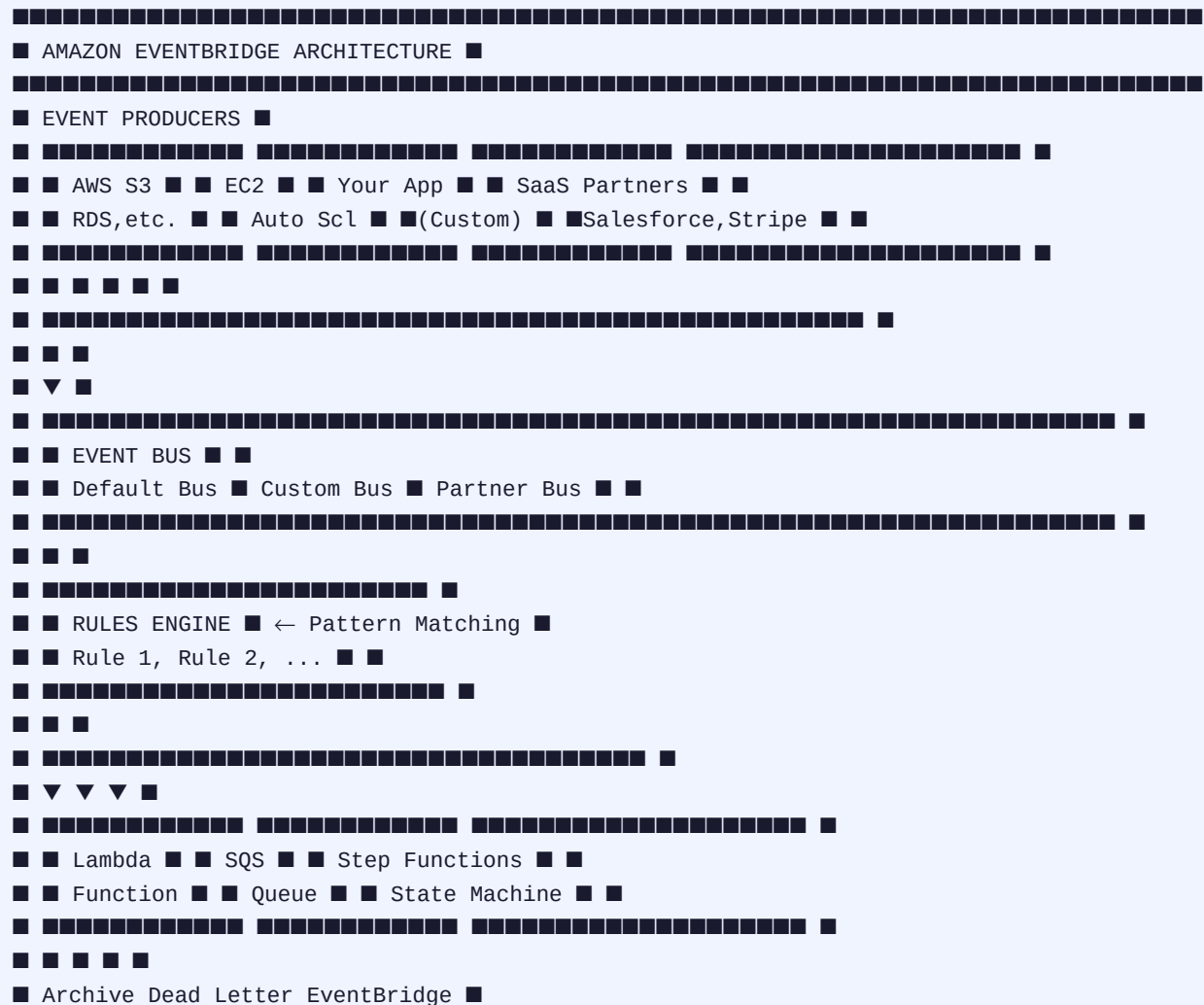
Bus Type	Description	Who Produces Events	Use Case
Partner Bus	Managed by SaaS partners	Zendesk, Shopify, Auth0, Stripe, etc.	SaaS integration

Event Rules & Pattern Matching

Rules use **event patterns** (JSON filters) to decide which events to route:

```
// Route only FAILED payments over $1000:
{
  "source": ["com.mycompany.payments"],
  "detail-type": ["Payment Processed"],
  "detail": {
    "status": ["FAILED"],
    "amount": [{ "numeric": [ ">", 1000] }]
  }
}
```

3. Architecture & Working





Step-by-Step Event Flow

Step	Action	Description
Step 1	Event is Produced	An AWS service, your app, or a SaaS partner generates an event (JSON object) when something happens.
Step 2	Event Arrives at Bus	The event is sent to the appropriate event bus (Default, Custom, or Partner).
Step 3	Rules Engine Evaluates	EventBridge checks each rule's event pattern against the incoming event. This happens in parallel, in milliseconds.
Step 4	Targets Are Invoked	All rules that match invoke their configured targets simultaneously. One event can trigger multiple targets.
Step 5	Retry on Failure	If a target fails, EventBridge retries with exponential backoff for up to 24 hours.
Step 6	Dead Letter Queue	If retries are exhausted, the event is sent to a configured Dead Letter Queue (DLQ) for analysis.
Step 7	Archive & Replay	All events are optionally archived. You can replay them later for debugging or reprocessing.

4. Features

Feature	What It Does
Schema Registry	Discovers and stores event schemas automatically. Generates code bindings in Python, Java, TypeScript. Enables auto-completion in IDEs.
Content-Based Filtering	Route events based on any field in the event payload using exact match, prefix, suffix, numeric ranges, IP ranges, and 'anything-but' patterns.
Event Transformation	Transform event payloads before sending to targets using Input Transformer — reshape JSON without writing Lambda code.
EventBridge Pipes	Point-to-point integrations with built-in filtering, enrichment via Lambda, and transformation between sources and targets.
Archive & Replay	Store all events indefinitely or for a set period. Replay archived events to any rule/target for debugging or migration.
Cross-Account/Region	Route events between AWS accounts and regions — critical for multi-account architectures and disaster recovery.
Global Endpoints	Replicate events across two regions for 99.99% availability. Automatic failover for business-critical workflows.
50+ Partner Integrations	Native connectors for Salesforce, Zendesk, Shopify, PagerDuty, Auth0, Datadog, and more — no custom code needed.

Feature	What It Does
Scheduling	Create scheduled rules (cron or rate expressions) to trigger targets on a schedule — like a managed cron job.
Resource-Based Policies	Fine-grained access control — specify which accounts/orgs can send events to your bus.

5. Real-World Use Cases

Industry	Icon	Use Case
E-Commerce	■	Order Processing Pipeline: Customer places order → EventBridge routes to Inventory (deduct stock), Payment service (charge card), Notification service (send confirmation email), and Analytics (log conversion) — all simultaneously.
Banking	■	Fraud Detection: Every transaction event is routed to a fraud ML model, compliance logger, and real-time dashboard. Suspicious events trigger an additional alert to security teams via SNS.
Healthcare	■	Patient Monitoring: IoT vitals devices emit events. EventBridge routes critical readings (heart rate > 150) to emergency alerts, doctor notifications, and EHR (Electronic Health Record) updates.
SaaS Integration	■	CRM Sync: When a deal is closed in Salesforce (Partner Event Source), EventBridge routes the event to update your internal billing system, provision the customer account, and trigger a welcome email workflow.
DevOps	■ ■	CI/CD Automation: CodePipeline failure event triggers a Lambda to notify the team on Slack, creates a Jira ticket, and rolls back the deployment via Step Functions.
AI/ML	■	Model Retraining: When new data arrives in S3, EventBridge triggers a Step Functions workflow to preprocess data, retrain the ML model on SageMaker, evaluate accuracy, and deploy if threshold is met.
Startup	■	Microservices Backbone: A startup's 10 microservices communicate entirely through EventBridge — no direct API calls, no complex service mesh. Each service publishes and subscribes to events independently.

6. Practical Example

Scenario: Build an order processing system where placing an order automatically triggers inventory update, email notification, and analytics logging.

Step 1: Create a Custom Event Bus

```
aws events create-event-bus --name order-processing-bus
```

Step 2: Publish an Order Event

```
aws events put-events --entries '[{
  "Source": "com.myshop.orders",
  "DetailType": "Order Placed",
```

```
"EventBusName": "order-processing-bus",
"Detail": "{\"orderId\":\"ORD-001\",\"amount\":150.00}"
}]'
```

Step 3: Create Rules

```
// Rule 1 - Notify inventory Lambda
aws events put-rule --name InventoryUpdate \
  --event-bus-name order-processing-bus \
  --event-pattern '{"source":["com.myshop.orders"],"detail-type":["Order Placed"]}'

// Add Lambda as target
aws events put-targets --rule InventoryUpdate \
  --event-bus-name order-processing-bus \
  --targets 'Id=1,Arn=arn:aws:lambda:us-east-1:123:function:update-inventory'
```

■ **Result:** When an order is placed, EventBridge simultaneously invokes the inventory Lambda, the notification SQS queue, and the analytics Kinesis stream — all with zero custom routing code.

7. Integrations

AWS Service	Role	Integration Pattern
AWS Lambda	Most common target	EventBridge invokes Lambda synchronously with event payload
Amazon SQS	Buffering target	Events are queued; Lambda or EC2 polls the queue
Amazon SNS	Fan-out target	Events fan out to multiple subscribers simultaneously
AWS Step Functions	Orchestration target	Complex multi-step workflows triggered by events
Amazon Kinesis	Streaming target	High-volume event streaming for analytics
Amazon API Gateway	HTTP target	Route events to REST/HTTP APIs
AWS CodePipeline	CI/CD source	Pipeline events trigger downstream automation
Amazon S3	Source & target	S3 events trigger rules; rules can write to S3
Amazon CloudWatch	Monitoring source	Alarm state changes trigger automated responses
AWS Glue	ETL target	Events trigger data processing jobs

8. Comparison

EventBridge vs similar AWS messaging/eventing services:

Feature	EventBridge	SNS	SQS	Kinesis
Type	Event Bus (Router)	Pub/Sub	Message Queue	Data Stream

Feature	EventBridge	SNS	SQS	Kinesis
Pattern Matching	■ Content-based filtering	■ Topic-based only	■ No filtering	■ No filtering
Max Targets	5 per rule	12.5M subscriptions	1 consumer per msg	Unlimited consumers
SaaS Integration	■ 50+ native partners	■ None	■ None	■ None
Event Replay	■ Built-in archive	■ No	■ No	■ 7-365 days
Schema Registry	■ Yes	■ No	■ No	■ No
Delivery	At-least-once	At-least-once	At-least-once	At-least-once
Latency	~0.5 seconds	~1-2 seconds	Near-real-time	~200ms
Use When	Routing events between services	Broadcasting notifications	Decoupling with buffering	Real-time data streaming

■ **Rule of Thumb:** Use EventBridge when you need smart event routing with content-based filtering. Use SNS for simple fan-out. Use SQS for reliable message queuing. Use Kinesis for high-throughput real-time streaming.

■ ■ AWS Step Functions

Visual Workflow Orchestration — Build, Run, and Visualize Complex Workflows

1. Simple Introduction

AWS Step Functions is a **serverless workflow orchestration service** that lets you coordinate multiple AWS services into visual, auditable workflows called **State Machines**. Instead of writing complex coordination code, you define your workflow as a series of steps in JSON or a visual editor.

■ **Real-World Analogy:** Think of Step Functions as an **assembly line manager in a factory**. The manager knows every step of production: receive raw materials → cut → weld → paint → quality check → package → ship. If any step fails, the manager knows exactly where to pause, retry, or reroute. The manager tracks every item and every step — nothing gets lost.

Business Problem Solved: Distributed applications need to coordinate many services in sequence. Without Step Functions, developers write fragile, untraceable 'spaghetti code' that's hard to debug and monitor. Step Functions provides a visual, reliable, and auditable way to manage complex workflows.

2. Core Concepts

Concept	Definition	Example
State Machine	The complete workflow definition (JSON/YAML)	An order fulfillment workflow with 8 steps
State	A single step in the workflow	ValidateOrder, ChargePayment, SendEmail
Transition	Moving from one state to the next	After payment succeeds → go to ShipOrder
Execution	A single run of the state machine	Processing Order #12345
Task State	Calls an AWS service or Lambda	Invoke Lambda to process payment
Choice State	Branches based on conditions (if/else)	If amount > \$1000 → require approval
Wait State	Pauses for a duration or timestamp	Wait 24 hours for confirmation
Parallel State	Runs multiple branches simultaneously	Process payment AND notify AND update inventory at once
Map State	Iterates over a list, runs steps for each item	Process 1000 orders in parallel
Pass State	Passes input to output, optionally transforming	Reshape JSON data between steps
Succeed/Fail	Terminal states ending execution successfully or with error	Workflow completed or failed with reason

Concept	Definition	Example
Catch/Retry	Error handling — retry failed steps or catch errors	Retry Lambda 3 times; if still fails → send alert
ASL	Amazon States Language — JSON-based workflow definition	The language Step Functions workflows are written in

[illegible]

State Machine Definition (ASL Example)

```
{
  "Comment": "Order Processing Workflow",
  "StartAt": "ValidateOrder",
  "States": {
    "ValidateOrder": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123:function:validate-order",
      "Retry": [{ "ErrorEquals": ["Lambda.ServiceException"], "MaxAttempts": 3 }],
      "Catch": [{ "ErrorEquals": ["States.ALL"], "Next": "OrderFailed" }],
      "Next": "CheckAmount"
    },
    "CheckAmount": {
      "Type": "Choice",
      "Choices": [{
        "Variable": "$.amount",
        "NumericGreaterThan": 1000,
        "Next": "RequireApproval"
      }],
      "Default": "ProcessPayment"
    },
    "OrderFailed": { "Type": "Fail", "Cause": "Validation Failed" }
  }
}
```

4. Features

Feature	Description	Benefit
Visual Workflow Designer	Drag-and-drop workflow builder in AWS Console	No code needed to design workflows
Built-in Error Handling	Retry and Catch on every state	Resilient workflows without custom code
SDK Integrations	Direct integration with 200+ AWS services — no Lambda needed	Invoke DynamoDB, S3, SageMaker directly
Optimistic Locking	Prevents concurrent execution conflicts	Safe parallel processing
Execution History	Complete audit trail of every step, input, output	Full observability and debugging

Feature	Description	Benefit
X-Ray Integration	Distributed tracing across all states	End-to-end performance visibility
Human Approval	Pause workflow and wait for human input (callback pattern)	Approval workflows, manual review steps
Map State	Distributed processing of lists — run 1000 items in parallel	High-throughput batch processing
Versioning & Aliases	Deploy new versions; route traffic between versions	Safe workflow deployments
AWS Glue & Athena SDK	Directly call Glue jobs and Athena queries	ETL pipelines without custom code

5. Real-World Use Cases

Industry	Use Case	Workflow Steps
E-Commerce	Order Fulfillment	Validate → Charge → Reserve Inventory → Ship → Notify
Banking	Loan Application	Apply → Credit Check → Verify Docs → Manager Approval → Disburse
Healthcare	Patient Onboarding	Register → Verify Insurance → Schedule → Send Welcome Kit
Media	Video Processing	Upload → Transcode (Parallel: HD/SD/Mobile) → Thumbnail → CDN Publish
AI/ML	ML Pipeline	Ingest Data → Preprocess → Train Model → Evaluate → Deploy if Accurate
DevOps	Blue/Green Deployment	Deploy Blue → Run Tests → If Pass: Switch Traffic → If Fail: Rollback
Finance	Trade Settlement	Validate Trade → Reserve Funds → Execute → Confirm → Audit Log

6. Practical Example

Scenario: Image processing pipeline — when a user uploads an image to S3, automatically resize it, add a watermark, generate a thumbnail, and notify the user.

```
// Triggered by S3 event via EventBridge
{
  "StartAt": "ResizeImage",
  "States": {
    "ResizeImage": {
      "Type": "Task",
      "Resource": "arn:aws:lambda::function:resize-image",
      "Next": "ParallelProcess"
    },
  },
}
```

```

    "ParallelProcess": {
      "Type": "Parallel",
      "Branches": [
        { "StartAt": "Watermark", "States": { "Watermark": { "Type": "Task", ... } } },
        { "StartAt": "Thumbnail", "States": { "Thumbnail": { "Type": "Task", ... } } }
      ],
      "Next": "NotifyUser"
    },
    "NotifyUser": {
      "Type": "Task",
      "Resource": "arn:aws:states:::sns:publish",
      "Parameters": { "TopicArn": "arn:aws:sns:...", "Message": "Image processed!" },
      "End": true
    }
  }
}

```

7. Integrations

Service	How It Integrates	Use Case
AWS Lambda	Invoke functions as Task states	Custom business logic at any workflow step
Amazon EventBridge	Trigger workflows from events	Event-driven workflow automation
Amazon DynamoDB	Read/write directly via SDK integration	Store workflow state and results
Amazon SageMaker	Trigger training, transform, endpoint creation	Automated ML pipelines
Amazon ECS/Fargate	Run containerized tasks as workflow steps	Long-running compute tasks
Amazon SNS/SQS	Publish notifications, send/receive messages	Decoupled communication in workflows
AWS Glue	Trigger ETL jobs	Data transformation workflows
Amazon Bedrock	Invoke LLM models	AI-powered workflow decisions
Amazon API Gateway	Invoke APIs and wait for callbacks	Third-party service integration
AWS CodeBuild	Trigger builds as part of CI/CD workflows	Automated deployment pipelines

8. Comparison

Feature	Step Functions	AWS Glue Workflows	Apache Airflow (MWAA)
Primary Use	General workflow orchestration	ETL/Data pipelines	Data engineering pipelines

Feature	Step Functions	AWS Glue Workflows	Apache Airflow (MWAA)
Visual Builder	■ Built-in	■ Basic	■ Code-based (DAGs)
Error Handling	■ Built-in Retry/Catch	■■ Limited	■ Yes (custom code)
AWS Integration	■ 200+ native SDK calls	■ Glue-focused	■■ Via operators
Scalability	Serverless, auto-scale	Managed	Managed (you size workers)
Human Approval	■ Callback pattern	■ No	■ Limited
Duration	Up to 1 year	Hours/days	Unlimited
Pricing	Per state transition	Per DPU hour	Per environment + instance
Best For	Application orchestration, business workflows	AWS data ETL	Complex data pipelines with dependencies

Amazon AppFlow

No-Code Data Integration — Move Data Between SaaS Apps and AWS Effortlessly

1. Simple Introduction

Amazon AppFlow is a **fully managed integration service** that enables you to securely transfer data between SaaS applications (like Salesforce, Zendesk, ServiceNow, Slack, Google Analytics) and AWS services (like S3, Redshift, RDS) — with **zero custom code**.

■ **Real-World Analogy:** AppFlow is like a **smart water pipeline system**. You tell it the source (a mountain spring = Salesforce), the destination (a reservoir = Amazon S3), how often to flow (every hour), and what to filter out (only water above quality grade A = filter records). The pipeline manages itself — you don't dig the trenches or maintain the pumps.

Business Problem: Data from SaaS apps is critical for analytics, compliance, and ML. But moving it requires building custom API integrations, handling pagination, rate limits, authentication, and transformation — all complex and expensive. AppFlow handles all of this for you.

2. Core Concepts

Concept	Definition	Example
Flow	A configuration that defines data transfer from source to destination	Transfer Salesforce Opportunities to S3 daily
Source Connector	The SaaS app or AWS service providing data	Salesforce, Zendesk, SAP, Google Analytics
Destination Connector	Where data is sent	Amazon S3, Redshift, Snowflake, Salesforce
Connection	Authenticated connection to a connector	OAuth connection to Salesforce org
Trigger	What starts the flow (schedule, event, on-demand)	Every day at 2 AM / when a new record is created
Field Mapping	Map source fields to destination fields	Salesforce 'Account.Name' → S3 column 'company_name'
Data Filter	Filter records before transferring	Only transfer Opportunities with Stage='Closed Won'
Data Transformation	Convert, mask, validate, or enrich data	Mask SSN fields, convert date formats
Incremental Transfer	Only transfer new/changed records	Transfer only records modified since last run
Private Link	Transfer data over AWS network, never the public internet	HIPAA-compliant data transfer

Supported Connectors

Category	Connectors
CRM	Salesforce, SAP, Zendesk, HubSpot, Marketo
Analytics	Google Analytics, Adobe Analytics, Amplitude
Productivity	Slack, ServiceNow, Asana, Jira, Veeva
Marketing	Mailchimp, ActiveCampaign, Pardot
E-Commerce	Shopify, BigCommerce
Finance	QuickBooks, Stripe
AWS Sources	S3, EventBridge
AWS Destinations	S3, Redshift, RDS Aurora, Salesforce, Snowflake, EventBridge, Upsolver

3. Architecture & Working



Flow Execution Steps

Step	What Happens
1. Trigger	Flow starts based on schedule, event, or manual trigger

Step	What Happens
2. Authenticate	AppFlow uses stored credentials/OAuth to connect to source
3. Extract	AppFlow calls source API, handles pagination automatically (up to 100GB per run)
4. Filter	Records not matching filter conditions are dropped
5. Transform	Fields are mapped, masked, validated, concatenated, or calculated
6. Encrypt	Data is encrypted using AWS KMS before transfer
7. Load	Data is written to destination in chosen format (JSON, CSV, Parquet)
8. Log	CloudWatch logs the run result; CloudTrail logs API calls

4. Features

Feature	Description
No-Code Configuration	Fully configured via console or API — no custom integration code
Bi-Directional Flows	Transfer data from AWS to SaaS and from SaaS to AWS
Data Masking	Automatically mask PII fields (SSN, credit card, email) during transfer
Incremental Refresh	Transfer only new/updated records using bookmarks — efficient and fast
Data Validation	Reject records that fail validation rules before loading
Custom Connectors SDK	Build custom connectors for internal or unlisted apps
EventBridge Integration	Publish flow completion events for downstream automation
Aggregation	Group and aggregate records during transfer (sum, count, etc.)
PrivateLink Support	Keep data on AWS network — never traverse the public internet
Up to 100GB Per Run	Handle large datasets per flow execution
Flow History	Audit log of every flow run with record counts and errors

5. Real-World Use Cases

Industry	Use Case
Marketing Analytics	Transfer Google Analytics and Marketo data to Redshift daily for unified marketing dashboards
Customer 360	Sync Salesforce CRM data to S3/Redshift for a complete customer view alongside transaction data
Compliance	Export Zendesk ticket data to S3 for 7-year retention required by compliance regulations

Industry	Use Case
Sales Forecasting	Sync Salesforce Opportunity data to SageMaker for ML-based revenue forecasting
Support Analytics	Transfer ServiceNow incident data to Redshift for SLA performance analysis
E-Commerce	Sync Shopify order data to Redshift for inventory forecasting and customer analysis
Healthcare	Transfer patient appointment data from Epic to S3 using PrivateLink for HIPAA compliance
Finance	Sync QuickBooks financial data to Redshift for consolidated reporting across entities

6. Practical Example

Scenario: Transfer Salesforce 'Closed Won' Opportunities to Amazon S3 daily as Parquet files for analytics.

```
// AWS CLI: Create AppFlow flow
aws appflow create-flow \
  --flow-name "salesforce-to-s3-daily" \
  --trigger-config '{"triggerType":"Scheduled","triggerProperties":{"ScheduleExpression":"rate(1days)","DataPullMode":"Incremental"}}' \
  --source-flow-config '{
    "connectorType": "Salesforce",
    "connectorProfileName": "my-salesforce-conn",
    "sourceConnectorProperties": {
      "Salesforce": {"object": "Opportunity", "enableDynamicFieldUpdate": true}
    }
  }' \
  --destination-flow-config-list '[{
    "connectorType": "S3",
    "destinationConnectorProperties": {
      "S3": {"bucketName": "my-analytics-bucket", "bucketPrefix":
"salesforce/opportunities",
      "s3OutputFormatConfig": {"fileType": "PARQUET"}}
    }
  }]' \
  --tasks '[{"taskType":"Filter","sourceFields":["StageName"],"connectorOperator":{"Salesforce":"EQUAL_TO"},"taskProperties":{"VALUE":"Closed Won"}},
    {"taskType":"Map","sourceFields":["Id","Name","Amount","CloseDate"],...}]'
```

7. Integrations

AWS Service	Integration
Amazon S3	Most common destination — store raw or transformed data as JSON, CSV, or Parquet
Amazon Redshift	Load SaaS data directly into Redshift tables for analytics
Amazon EventBridge	Publish flow completion events; use EventBridge events as flow triggers

AWS Service	Integration
AWS KMS	Encrypt data at rest using customer-managed keys
AWS CloudTrail	Audit all AppFlow API calls for compliance
Amazon CloudWatch	Monitor flow run metrics, errors, and durations
AWS Glue	Catalog AppFlow data in S3 for Athena/EMR querying
Amazon SageMaker	Feed SaaS data to ML models via S3
AWS PrivateLink	Secure data transfer over private AWS network

8. Comparison

Feature	AppFlow	AWS Glue	AWS DMS	Fivetran/Stitch
Type	SaaS Integration	ETL/ELT	DB Migration	3rd-party ETL
No-Code	■ Yes	■■ Visual ETL	■■ Partial	■ Yes
SaaS Connectors	■ 50+ native	■ No	■ DBs only	■ 200+
Data Volume	Up to 100GB/run	Unlimited	Unlimited	Unlimited
DB-to-DB	■ No	■ Yes	■ Yes	■ Yes
Cost Model	Per flow + GB	Per DPU	Per instance	Per row/credit
AWS Native	■ Yes	■ Yes	■ Yes	■ No
Best For	SaaS → AWS data sync	Complex ETL transforms	DB migrations	SaaS → DW (any cloud)

■ Amazon SNS

Simple Notification Service — Broadcast Messages to Millions Instantly

1. Simple Introduction

Amazon SNS (Simple Notification Service) is a **fully managed pub/sub messaging service** that enables you to send messages to many recipients simultaneously. One publisher sends a message; all interested subscribers receive it instantly.

■ **Real-World Analogy:** SNS is like a **radio broadcast station**. The station (publisher) broadcasts a signal (message) on a frequency (topic). Anyone who tunes in (subscribes) receives the broadcast. The station doesn't know or care how many listeners there are — it just broadcasts. Listeners can be on FM (Lambda), AM (SQS), or shortwave (HTTP/email).

Business Problem: Applications need to notify multiple systems simultaneously when something happens — without knowing who the receivers are, how many there are, or building point-to-point connections to each.

2. Core Concepts

Concept	Definition	Example
Topic	A named channel to which messages are published	'order-notifications', 'system-alerts'
Publisher	Any entity that sends a message to a topic	Lambda function, EC2, mobile app, AWS service
Subscriber	Receives messages published to a topic	Lambda, SQS queue, HTTP endpoint, email
Subscription	A registered endpoint on a topic	SQS queue ARN subscribed to 'order-notifications'
Message	The content sent to a topic (up to 256KB)	JSON payload with order details
Message Attribute	Metadata attached to a message for filtering	'category': 'electronics', 'region': 'us-east'
Subscription Filter Policy	JSON filter — subscribers only get matching messages	Only receive messages where category=electronics
Fan-out Pattern	One SNS message triggers multiple SQS queues/Lambdas	One order event → Inventory + Billing + Notification queues
Message Deduplication	Prevent duplicate deliveries (FIFO topics)	Same message ID won't be delivered twice

Concept	Definition	Example
Dead Letter Queue	Catches messages that couldn't be delivered	Failed email deliveries stored in SQS DLQ

SNS Topic Types

Type	Standard Topic	FIFO Topic
Message Order	Not guaranteed	Strictly ordered
Deduplication	Possible duplicates	Exactly-once (dedup ID)
Throughput	Nearly unlimited (300M/month free)	300 TPS (transactions per second)
Subscribers	Lambda, SQS, HTTP, Email, SMS, Mobile Push	SQS FIFO queues only
Use Case	High-throughput notifications, alerts	Financial transactions, order updates requiring order
Cost	Free tier: 1M requests/month	Slightly higher

Subscription Protocols

Protocol	Description	Use Case
AWS Lambda	Invoke Lambda function with message	Process and react to messages with custom code
Amazon SQS	Send message to SQS queue (fan-out)	Buffer messages for reliable processing
HTTP/HTTPS	POST message to an HTTP endpoint	Webhooks to external services
Email/Email-JSON	Send notification email	Human notification, alerting teams
SMS	Send text message	Mobile alerts, OTP delivery, emergency notifications
Mobile Push	Push to iOS (APNs), Android (FCM), FireOS	Mobile app push notifications
Amazon Kinesis	Stream messages to Kinesis Firehose	Real-time analytics pipelines

3. Architecture & Working



SNS Fan-Out Pattern

4. Features

Feature	Description
Message Filtering	Each subscriber defines a filter policy — receives only relevant messages, reducing cost and noise
Message Encryption	SSE (Server-Side Encryption) with AWS KMS for sensitive messages

Feature	Description
Mobile Push Notifications	Native support for APNs (Apple), FCM (Google/Android), ADM (Amazon), Baidu
SMS Messaging	Send SMS to 200+ countries; supports OTP, alerts, marketing messages
FIFO Topics	Ordered, exactly-once delivery for critical business events
Raw Message Delivery	Deliver unmodified message payload (no SNS envelope wrapper) to SQS/HTTP
Large Message Support	Messages up to 256KB; use Extended Client Library with S3 for larger
Cross-Region Delivery	SNS topics in one region can deliver to subscribers in other regions
Dead Letter Queues	Capture undeliverable messages for investigation and reprocessing
CloudWatch Monitoring	Monitor delivery success rates, failures, and latencies per subscription

5. Real-World Use Cases

Industry	Use Case
E-Commerce	Order confirmation: customer gets email, inventory system gets SQS message, analytics gets Lambda event — all from one SNS publish
Banking	Fraud alert: one SNS publish notifies compliance team via email, triggers Lambda to freeze account, and pages on-call team via SMS simultaneously
Healthcare	Critical lab result: alerts doctor via SMS, updates patient portal via Lambda, logs to compliance system via SQS
DevOps	CI/CD alerts: failed deployment triggers SNS which emails the team, posts to Slack via Lambda, and creates a PagerDuty alert via HTTP
Mobile Apps	Push notifications: send personalized push notifications to millions of iOS/Android devices for offers, alerts, or updates
IoT	Device alerts: temperature sensor exceeds threshold → SNS alerts maintenance team, triggers auto-shutdown Lambda, logs to DynamoDB
News Media	Breaking news: publish one event, deliver to web subscribers (HTTP), mobile users (push), and email list simultaneously

6. Practical Example

Scenario: E-commerce order notification system — one order event fans out to email, inventory, and analytics simultaneously.

```
import boto3, json

sns = boto3.client('sns', region_name='us-east-1')

# 1. Create Topic
```

```

topic = sns.create_topic(Name='order-events')
topic_arn = topic['TopicArn']

# 2. Subscribe SQS Queue (Inventory Service)
sns.subscribe(TopicArn=topic_arn, Protocol='sqs',
    Endpoint='arn:aws:sqs:us-east-1:123:inventory-queue',
    Attributes={'FilterPolicy': json.dumps({"eventType": ["ORDER_PLACED"]})})

# 3. Subscribe Lambda (Analytics)
sns.subscribe(TopicArn=topic_arn, Protocol='lambda',
    Endpoint='arn:aws:lambda:us-east-1:123:function:order-analytics')

# 4. Subscribe Email (Customer notification)
sns.subscribe(TopicArn=topic_arn, Protocol='email',
    Endpoint='orders@myshop.com')

# 5. Publish Order Event
sns.publish(
    TopicArn=topic_arn,
    Subject='New Order Placed',
    Message=json.dumps({"orderId":"ORD-001", "amount":299.99, "customer":"John Doe"}),
    MessageAttributes={"eventType": {"DataType":"String", "StringValue":"ORDER_PLACED"}}
)

```

7. Integrations

Service	Integration
Amazon SQS	Fan-out: SNS delivers to multiple SQS queues for reliable, independent processing
AWS Lambda	Invoke Lambda functions for custom message processing logic
Amazon EventBridge	SNS can be an EventBridge target; EventBridge can publish to SNS
Amazon S3	S3 events published to SNS for fan-out to multiple processors
Amazon CloudWatch	Alarms publish to SNS topics for alerting
AWS CloudFormation	Stack events notify SNS for deployment monitoring
Amazon RDS	RDS events (failover, backup) notify operations teams via SNS
AWS CodePipeline	Pipeline stage transitions notify teams via SNS
Amazon Kinesis Firehose	SNS can deliver to Kinesis for real-time analytics

8. Comparison

Feature	SNS	SQS	EventBridge
Pattern	Pub/Sub (Push)	Queue (Pull)	Event Bus (Route)

Feature	SNS	SQS	EventBridge
Delivery Model	Push to all subscribers	Pull by consumer	Route by rules to targets
Message Retention	No storage (push only)	Up to 14 days	Up to 1 year (archive)
Filtering	Simple attribute-based	None (at receipt)	Content-based (complex)
Fan-out	■ Native (many subscribers)	■ 1 consumer per message	■ Many targets per rule
Ordering	FIFO topics only	FIFO queues	Ordered within bus
SMS/Push	■ Native	■ No	■ No
SaaS Sources	■ No	■ No	■ 50+ partners
Best For	Broadcasting to many	Reliable task queuing	Smart event routing

■ Amazon SQS

Simple Queue Service — Reliable Message Buffering Between Services

1. Simple Introduction

Amazon SQS (Simple Queue Service) is a **fully managed message queuing service** that enables you to decouple and scale microservices, distributed systems, and serverless applications. Messages are stored in a queue until a consumer retrieves and processes them.

■ **Real-World Analogy:** SQS is like a **physical queue at a government office**. When you arrive (producer sends a message), you take a ticket and wait in line (queue). The officer (consumer) calls the next ticket, serves that person (processes the message), and moves on. If the officer is sick (consumer fails), no one leaves — they wait. The queue ensures no request is lost even if the service is temporarily unavailable.

Business Problem: In distributed systems, producers and consumers operate at different speeds. Without a queue, a fast producer can overwhelm a slow consumer. SQS acts as a buffer — the producer keeps sending, the consumer processes at its own pace, and no messages are lost.

2. Core Concepts

Concept	Definition	Example
Queue	A temporary storage buffer for messages	'order-processing-queue'
Message	Data sent to a queue (up to 256KB)	JSON order object
Producer	Sends messages to the queue	API server sending order events
Consumer	Receives and processes messages from the queue	Lambda function processing orders
Visibility Timeout	Time a message is hidden after being received (processing time)	30 seconds — consumer must delete within this time
Message Retention	How long messages stay in queue if not processed	Default 4 days, max 14 days
Dead Letter Queue (DLQ)	Separate queue for messages that fail processing repeatedly	Failed orders after 3 retries go to 'failed-orders-dlq'
Long Polling	Consumer waits up to 20 seconds for a message (reduces empty responses)	Efficient polling — don't check empty queue constantly
Short Polling	Consumer returns immediately (even if empty queue)	Fast but costly — many empty API calls

[illegible]

Message Lifecycle

Phase	Description
1. Send	Producer calls SendMessage API — message stored in queue
2. Receive	Consumer polls queue; message becomes invisible (visibility timeout starts)
3. Process	Consumer processes the message — business logic runs
4. Delete	Consumer calls DeleteMessage API — message permanently removed
5. Retry	If consumer fails to delete within visibility timeout, message reappears in queue
6. DLQ	After maxReceiveCount retries, message moves to Dead Letter Queue

4. Features

Feature	Description
Server-Side Encryption	SSE with AWS KMS — messages encrypted at rest
VPC Endpoints	Access SQS from within VPC without internet — no data leaves AWS network
Long Polling	Wait up to 20 seconds for message — reduces API calls and cost by 90%+
Message Batching	Send/receive/delete up to 10 messages in one API call — reduces cost
Dead Letter Queue	Automatically move failed messages to DLQ after N receive attempts
Delay Queues	Delay message delivery from 0 to 900 seconds (15 minutes)
Message Timer	Override queue delay for individual messages
Extended Client Library	Store payloads > 256KB in S3; SQS stores reference pointer
CloudWatch Alarms	Trigger auto-scaling based on queue depth (messages waiting)
Lambda Triggers	Lambda polls SQS automatically — serverless message processing

Feature	Description
Resource Policies	Cross-account access — allow other accounts to send/receive

5. Real-World Use Cases

Industry	Use Case
E-Commerce	Order processing queue: web servers send orders to SQS; backend Lambda processes them at its own pace — handles Black Friday traffic spikes without dropping orders
Video Streaming	Video encoding queue: uploaded videos go to SQS; EC2 workers pick them up for transcoding into multiple resolutions (480p, 720p, 1080p)
Banking	Transaction processing: all transactions queued in SQS FIFO; processing happens exactly-once and in strict order per account
Healthcare	Lab result processing: test results queued in SQS; multiple workers process them — some for patient records, some for doctor alerts, some for billing
Email Marketing	Email send queue: 10M marketing emails queued; SES workers consume at allowed rate (500/sec) without throttling or losing emails
DevOps	Build queue: developer pushes code → job added to SQS → available build agents pick up and run → results logged
IoT	Sensor data buffer: 10,000 IoT devices send readings to SQS; analytics backend processes them when capacity is available
Gaming	Leaderboard updates: score submissions queued in SQS; single writer processes them in order to avoid race conditions on leaderboard

6. Practical Example

Scenario: Order processing system with a Dead Letter Queue for failed orders.

```
import boto3, json

sqs = boto3.client('sqs', region_name='us-east-1')

# 1. Create DLQ first
dlq = sqs.create_queue(QueueName='order-failures-dlq')
dlq_arn = sqs.get_queue_attributes(
    QueueUrl=dlq['QueueUrl'], AttributeNames=['QueueArn'])['Attributes']['QueueArn']

# 2. Create main queue with DLQ configured
queue = sqs.create_queue(
    QueueName='order-processing-queue',
    Attributes={
        'VisibilityTimeout': '30', # 30 sec to process
        'MessageRetentionPeriod': '86400', # 1 day retention
        'ReceiveMessageWaitTimeSeconds': '20', # Long polling
        'RedrivePolicy': json.dumps({
```

```

        'deadLetterTargetArn': dlq_arn,
        'maxReceiveCount': 3 # Move to DLQ after 3 failures
    })
}
)

# 3. Producer: Send order
sqs.send_message(
    QueueUrl=queue['QueueUrl'],
    MessageBody=json.dumps({"orderId":"ORD-001", "amount":150.00}),
    DelaySeconds=0
)

# 4. Consumer: Receive and process
msgs = sqs.receive_message(QueueUrl=queue['QueueUrl'], MaxNumberOfMessages=10)
for msg in msgs.get('Messages', []):
    order = json.loads(msg['Body'])
    print(f"Processing order: {order['orderId']}")
    # ... process order ...
    sqs.delete_message(QueueUrl=queue['QueueUrl'], ReceiptHandle=msg['ReceiptHandle'])

```

7. Integrations

Service	Integration
AWS Lambda	Lambda polls SQS automatically and processes messages in batches
Amazon SNS	Fan-out: SNS delivers to multiple SQS queues for parallel, decoupled processing
Amazon EventBridge	EventBridge routes events to SQS queues as a target
Amazon EC2 Auto Scaling	Scale EC2 consumers based on SQS queue depth (ApproximateNumberOfMessages)
Amazon S3	S3 events published to SQS for downstream processing
AWS Step Functions	SQS queues used for async task handoff between workflow steps
Amazon Kinesis	Kinesis → Lambda → SQS for fan-out of streaming data
AWS CloudWatch	Monitor queue depth, age of oldest message, DLQ depth
Amazon SES	Email send notifications (delivery, bounce, complaint) sent to SQS

8. Comparison

Feature	SQS	SNS	Kinesis Data Streams	RabbitMQ (MQ)
Type	Message Queue	Pub/Sub	Data Stream	Message Broker
Message Model	Pull (consumer polls)	Push (to subscriber)	Pull (shard consumer)	Push or Pull

Feature	SQS	SNS	Kinesis Data Streams	RabbitMQ (MQ)
Retention	Up to 14 days	None (push only)	1-365 days	Configurable
Order	FIFO queues only	FIFO topics only	Per shard ordered	Per queue
Throughput	Unlimited	Unlimited	1MB/s per shard	Limited
Replay	■ No	■ No	■ Yes	■ No
Multiple Consumers	No (1 consumer per msg)	Yes (fan-out)	Yes (per shard)	Yes
Best For	Task queuing, decoupling	Broadcasting notifications	Real-time streaming	Complex routing protocols

Amazon MWAA

Managed Workflows for Apache Airflow — Enterprise Data Pipeline Orchestration

1. Simple Introduction

Amazon MWAA (Managed Workflows for Apache Airflow) is a **fully managed service for Apache Airflow** — the popular open-source workflow orchestration platform used by data engineers worldwide. MWAA handles all the infrastructure so you can focus on building data pipelines.

■ **Real-World Analogy:** Apache Airflow is like a **railway network controller**. Multiple trains (workflows/DAGs) run on specific schedules along defined routes (task dependencies). Each train has a schedule, stops (tasks), and rules about what to do if a stop fails (retry). MWAA is like hiring a company to manage all the rails, signals, and station infrastructure — you just design the routes.

Business Problem: Data engineering teams need to orchestrate complex data pipelines with hundreds of interdependent tasks that run on schedules. Setting up, scaling, and maintaining Apache Airflow requires significant DevOps expertise. MWAA eliminates this operational burden.

2. Core Concepts

Concept	Definition	Example
Apache Airflow	Open-source platform to programmatically author, schedule, and monitor workflows	Used by Airbnb, LinkedIn, Twitter for data pipelines
DAG	Directed Acyclic Graph — a Python script defining tasks and their dependencies	etl_pipeline.py with 12 tasks
Task	A single unit of work in a DAG	extract_from_s3, transform_data, load_to_redshift
Operator	A template defining how a task runs	PythonOperator, BashOperator, S3ToRedshiftOperator
Scheduler	Parses DAGs and triggers task instances on schedule	Runs every 5 minutes to check for due tasks
Executor	Runs task instances — determines where/how tasks run	CeleryExecutor for distributed, LocalExecutor for single
Worker	Actually runs the task code	EC2 or Fargate instance running your Python task
Webserver	Airflow UI — monitor, trigger, debug DAGs	Web dashboard at your MWAA environment URL
Metadata DB	Stores DAG definitions, task states, logs	Aurora PostgreSQL managed by AWS

Concept	Definition	Example
Environment	An MWAA deployment (webserver + scheduler + workers)	my-data-team-airflow-env
DAG Run	A specific execution of a DAG at a point in time	etl_pipeline running for 2025-06-15
Task Instance	A specific run of a task within a DAG run	extract_from_s3 at 2025-06-15 02:00 AM
XCom	Cross-communication — pass data between tasks	Task A outputs file path; Task B reads it
Connection	Stored credential for external systems	my-redshift-conn, my-s3-conn, my-snowflake-conn
Variable	Key-value store for Airflow configuration	env=production, batch_size=1000
Pool	Limit concurrent task executions	Max 5 Redshift tasks at once (avoid overload)

[illegible]

Sample DAG (Python)

```
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.operators.glue import GlueJobOperator
from airflow.providers.amazon.aws.transfers.s3_to_redshift import S3ToRedshiftOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data-team',
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
    'email_on_failure': True,
    'email': ['data-alerts@company.com']
}

with DAG('daily_etl_pipeline', default_args=default_args,
        schedule_interval='0 2 * * *', # Run at 2 AM daily
        start_date=datetime(2025, 1, 1), catchup=False) as dag:

    extract = GlueJobOperator(task_id='extract_from_s3',
                              job_name='extract-raw-data', aws_conn_id='aws_default')

    transform = GlueJobOperator(task_id='transform_data',
                                 job_name='transform-clean-data', aws_conn_id='aws_default')

    load = S3ToRedshiftOperator(task_id='load_to_redshift',
                                 schema='analytics', table='daily_sales',
                                 s3_bucket='my-data-lake', s3_key='transformed/daily_sales/',
                                 redshift_conn_id='redshift_default', copy_options=['CSV'])

    def notify_success():
        print("ETL Pipeline completed successfully!")

    notify = PythonOperator(task_id='notify_team', python_callable=notify_success)

    # Define dependencies (task order)
```

```
extract >> transform >> load >> notify
```

4. Features

Feature	Description
Fully Managed Airflow	AWS manages webserver, scheduler, workers, metadata DB, upgrades, patches
Auto-Scaling Workers	Workers scale based on queue depth — handle variable workloads automatically
Apache Airflow UI	Full Airflow web UI for DAG monitoring, triggering, debugging, and log viewing
S3 for DAGs	Upload DAG files to S3 — MWAA automatically syncs and deploys them
VPC Integration	Deploy in your VPC — workers access private resources (RDS, EMR, Redshift)
IAM Integration	Airflow connections use IAM roles — no credentials stored in DAGs
Custom Plugins	Upload custom operators, sensors, and hooks as plugins to S3
Python Requirements	Install any Python package via requirements.txt uploaded to S3
Secrets Manager Integration	Store sensitive connections in AWS Secrets Manager — not in Airflow DB
Multi-AZ	Scheduler and webserver deployed across multiple Availability Zones
CloudWatch Logs	All DAG, task, scheduler, and webserver logs sent to CloudWatch
Airflow Version Support	Latest stable Airflow versions — upgrade without infrastructure work

5. Real-World Use Cases

Industry	Use Case
E-Commerce Analytics	Daily ETL: Extract orders from RDS → Transform with Glue → Load to Redshift → Update QuickSight dashboard
Financial Reporting	End-of-day pipeline: Aggregate transactions from 50 microservices → Reconcile → Generate P&L; reports → Email to CFO
Healthcare Data	HIPAA ETL: Extract HL7 data from EHR → Transform to FHIR format → Load to data lake → Train predictive models on SageMaker
Retail Forecasting	Weekly demand forecasting: Pull sales history from Redshift → Train forecast model on SageMaker → Store predictions in DynamoDB → Alert buyers
Media Processing	Content pipeline: Pull content from CMS → Translate to 20 languages → Generate thumbnails → Publish to CDN → Notify editors
Gaming Analytics	Player behavior: Stream events → Aggregate daily → Calculate engagement scores → Update player segments → Trigger personalized campaigns

Industry	Use Case
Banking Compliance	Regulatory reporting: Run nightly calculations across 10M accounts → Validate totals → Generate regulatory reports → Submit to regulators

6. Practical Example

Scenario: Set up an MWAA environment and deploy a DAG.

```
# Step 1: Create S3 bucket for DAGs
aws s3 mb s3://my-mwaa-dags-bucket
aws s3api put-object --bucket my-mwaa-dags-bucket --key dags/

# Step 2: Upload your DAG file
aws s3 cp daily_etl.py s3://my-mwaa-dags-bucket/dags/

# Step 3: Upload requirements (if needed)
echo "apache-airflow-providers-amazon>=8.0.0" > requirements.txt
aws s3 cp requirements.txt s3://my-mwaa-dags-bucket/

# Step 4: Create MWAA Environment (via console or CLI)
aws mwaa create-environment \
  --name my-data-team-airflow \
  --airflow-version 2.8.1 \
  --environment-class mw1.medium \
  --max-workers 10 \
  --dag-s3-path dags/ \
  --requirements-s3-path requirements.txt \
  --source-bucket-arn arn:aws:s3:::my-mwaa-dags-bucket \
  --execution-role-arn arn:aws:iam::123456789:role/mwaa-execution-role \
  --network-configuration '{"SubnetIds":["subnet-abc", "subnet-def"], "SecurityGroupIds":["sg-xyz"]}' \
  --logging-configuration '{
    "DagProcessingLogs":{"Enabled":true,"LogLevel":"INFO"},
    "TaskLogs":{"Enabled":true,"LogLevel":"INFO"},
    "WorkerLogs":{"Enabled":true,"LogLevel":"WARNING"}
  }'

# Step 5: Access Airflow UI
aws mwaa create-web-login-token --name my-data-team-airflow
```

7. Integrations

Service	Airflow Operator/Hook	Use Case
AWS Glue	GlueJobOperator, GlueDataQualityOperator	Trigger ETL jobs as DAG tasks
Amazon Redshift	S3ToRedshiftOperator, RedshiftSQLOperator	Load and query data warehouse

Service	Airflow Operator/Hook	Use Case
Amazon EMR	EmrCreateJobFlowOperator, EmrAddStepsOperator	Run Spark/Hadoop jobs at scale
Amazon S3	S3CopyObjectOperator, S3ListOperator, S3KeySensor	Read, write, move files in S3
Amazon SageMaker	SageMakerTrainingOperator, SageMakerTransformOperator	Train and run ML models
AWS Lambda	LambdaInvokeFunctionOperator	Run Lambda for lightweight processing
Amazon DynamoDB	DynamoDBToS3Operator	Export DynamoDB tables
Amazon RDS/Aurora	PostgresOperator, MySqlOperator	Run SQL queries on relational DBs
AWS Step Functions	StepFunctionStartExecutionOper ator	Trigger complex orchestrated workflows
Amazon Athena	AthenaOperator	Run serverless SQL queries on S3 data
AWS Secrets Manager	SecretsManagerBackend	Retrieve connection credentials securely

8. Comparison

Feature	MWAA (Airflow)	AWS Step Functions	AWS Glue Workflows	AWS CodePipeline
Primary Use	Data pipelines, ETL orchestration	App workflow orchestration	AWS Glue ETL pipelines	CI/CD pipelines
Language	Python (DAGs)	JSON (ASL)	GUI/API	GUI/YAML
Data Focus	■ Strong (Spark, SQL, ML)	■■ General purpose	■ Glue-centric	■ Deployment focus
Scheduling	■ Cron-based (powerful)	■■ Via EventBridge	■■ Triggers/schedules	■■ Event-triggered
Error Handling	■ Retry, alerting, backfill	■ Built-in Retry/Catch	■■ Limited	■■ Stage-level only
Observability	■ Rich UI, Gantt charts	■ Visual execution graph	■■ Basic	■ Pipeline view
Open Source	■ Apache Airflow	■ AWS proprietary	■ AWS proprietary	■ AWS proprietary
Learning Curve	High (Python + Airflow)	Medium (JSON + AWS)	Low (GUI)	Low (GUI)
Managed	■ Fully managed	■ Serverless	■ Fully managed	■ Fully managed
Best For	Complex data eng pipelines	Business workflows	AWS-only ETL	App deployment

■ **Choosing the Right Orchestrator:** Use **MWAA** for complex data engineering with many dependencies, backfilling, and Python ecosystem. Use **Step Functions** for business process workflows and AWS service orchestration. Use **Glue Workflows** for simple Glue-only pipelines. Use **CodePipeline** for CI/CD.

Service Summary & Quick Reference

Service	Type	Primary Use	Key Strength	When to Use
EventBridge ■	Event Bus	Event routing	Content-based filtering + SaaS sources	Smart event routing between services
Step Functions ■■	Orchestrator	Workflow automation	Visual workflows + 200 AWS SDK integrations	Multi-step business workflows
AppFlow ■	Integration	SaaS-to-AWS data sync	No-code, 50+ SaaS connectors	Move SaaS data to AWS for analytics
SNS ■	Pub/Sub	Fan-out notifications	Push to millions simultaneously	Broadcasting to many subscribers
SQS ■	Queue	Message buffering	Reliable at-least-once delivery + DLQ	Decoupling services, absorbing spikes
MWAA ■	Orchestrator	Data pipeline orchestration	Full Apache Airflow + managed infra	Complex data engineering pipelines

Integration Patterns at a Glance

Most Common Integration Patterns:

1. EVENT-DRIVEN MICROSERVICES:

Your App → EventBridge → [Lambda, SQS, Step Functions] (fan-out via rules)

2. RELIABLE TASK PROCESSING:

Producer → SNS Topic → [SQS Queue 1, SQS Queue 2, Lambda] (fan-out)

→ Each SQS consumed independently by different services

3. DATA PIPELINE:

SaaS App → AppFlow → S3 → (MWAA triggers) → Glue ETL → Redshift

4. COMPLEX BUSINESS WORKFLOW:

EventBridge → Step Functions → [Lambda, DynamoDB, SNS, SQS, SageMaker]

5. HIGH-THROUGHPUT PROCESSING:

S3 Upload → EventBridge → SQS → Lambda (auto-scaled) → DynamoDB

6. APPROVAL WORKFLOW:

Request → Step Functions → Human Approval (wait) → SNS Alert

→ Approve/Reject → Next Step